

國立臺灣科技大學電機工程系

114 學年度第一學期

專題成果報告

基於權重特性之低成本高性能人工智慧
加速器架構設計

組 別: D05-1141

組 員: 姓名: _____ 林明宏 _____ 學號: _____ B11107157 _____

姓名: _____ 蕭家宏 _____ 學號: _____ B11130007 _____

指導老師: 呂學坤 (簽名)

中華民國 114 年 11 月 27 日

目錄

一、緒論.....	1
1.1 研究背景.....	1
1.2 研究動機.....	2
1.3 文獻回顧.....	3
1.4 專題貢獻.....	6
二、理論背景.....	7
2.1 張量處理器 [1].....	7
2.2 Data Flow (WS/OS) [5]	8
2.3 訓練後量化 (Post-Training Quantization, PTQ).....	10
2.4 Most Significant Runs (MSRs) [6]	11
2.5 期望值補償 [2].....	11
三、實驗數據.....	13
3.1 實驗模型介紹.....	13
3.2 權重 MSR 分布分析	14
3.3 權重儲存與補償方式.....	16
3.4 所提出之張量處理器架構.....	18
3.4.1 Memory Wrapper	19
3.4.2 權重預處理單元 (WPU).....	20
3.4.3 縮減精度處理單元 (RPE).....	22
3.4.4 補償精度處理單元 (CPE).....	23
3.4.5 輸入緩衝器 (Input Buffer).....	24
3.4.6 累加器 (Accumulator)	25
3.4.7 激活函數 (Activation Function).....	26
3.4.8 重新量化 (Re-quantization / Normalization)	26
3.4.9 張量處理器系統控制器 (TPU System Controller)	27
3.5 電路模擬.....	28
3.5.1 權重預處理單元電路模擬.....	28
3.5.2 RPE 電路模擬	29
3.5.3 CPE 電路模擬	29
3.5.4 張量處理器電路模擬.....	30
3.6 硬體開銷分析與超大型積體電路實現結果.....	31

目錄

3.7 準確度分析.....	34
3.8 性能分析.....	34
四、建議與結論.....	35
五、未來發展方向.....	36
5.1 Block Floating Point (BFP)	36
5.2 Microsoft Floating Point (MSFP) [11].....	37
六、時間進度表.....	38
七、參考資料.....	39
八、工作分配.....	41
九、經費表.....	42

一、緒論

1.1 研究背景

現今的人工智慧 (Artificial Intelligence, AI) 發展迅速，尤其是在深度學習 (Deep Learning) 的領域中被廣泛運用，其對於運算資源的需求也日益提升。從語音辨識、影像處理到自然語言的理解，這些應用往往需要大量的矩陣運算，對於傳統的中央處理器 (Central Processing Unit, CPU) 與影像的圖形處理器 (Graph Processing Unit, GPU) 來說，在面對深度學習應用這種規模的任務時，可能會面臨性能瓶頸和能源效率的問題。在這樣的背景下，人工智慧加速器 (AI Accelerator) 應運而生，作為專為機器學習與深度神經網路計算而設計的高效能運算硬體裝置，能大幅提升運算效能與能源效率。

其中，Google 所推出的張量處理器 (Tensor Processing Unit, TPU) [1] 是具代表性的一種專用人工智慧加速器，用於加快機器學習工作負載的處理速度，並會使用專為執行機器學習演算法中常見的大型矩陣運算而設計的硬體，更有效率地訓練模型。張量處理器的主要工作為矩陣處理，也就是乘法與累加運算的組合，透過針對張量運算的硬體優化，能實現比傳統中央處理器更高的運算效能。

張量處理器的核心：**脈動陣列 (Systolic Array) + 流水線 (Pipeline)**。脈動陣列是一種硬體設計架構，由相互連接的處理單元 (Processing Element, PE) 排成二維網狀所組成(如 128 x 128 或 256 x 256)，每個處理單元會透過本身的乘積累加單元 (Multiply Accumulate Unit, MAC Unit) 執行少量乘加運算，並會將結果傳遞至相鄰的處理單元。因此，當數據資料進入此架構時，就不需要額外的控制邏輯來處理數據的進出，也大幅地減少了處理單元頻繁訪問晶片外記憶體的需求。而此架構也十分適合執行 pipeline 運算，可以同時執行乘加運算，以實現極高的計算密度和吞吐量。

1.2 研究動機

傳統上，人工智慧主要基於雲端，資料會傳送到資料中心，在那邊處理並傳回經過分析後的資料。而邊緣人工智慧 (AI at the Edge, Edge AI) 則是將人工智慧的模型推論 (Inference) 與決策能力，從集中式的雲端資料中心，向下帶入至資料生成的源頭 (即終端裝置) 的一種分散式運算形式，以實現快速的資料分析與獨立於雲端或資料中心的行動。這種架構的轉變，是為了反應傳統雲端人工智慧在物聯網 (IoT) 時代所面臨的幾個挑戰。

首先是**延遲性 (Latency)** 的需求，許多企業在推論時需要高速資料分析的能力，通常需要即時處理資料，因此對於延遲性要求嚴苛，而在傳統雲端人工智慧，資料在往返傳輸期間可能會遺失其封包或產生延遲，進而造成錯誤。舉例來說，在自動駕駛、遠端手術或工業即時控制等對時間高度敏感的應用中，數據往返雲端所耗費的數百毫秒延遲，可能會導致系統失效甚至帶來安全風險。雲端運算無法保證毫秒級的即時決策，因此邊緣人工智慧的出現，可以實現接近零延遲的本地端判斷。

再來是**成本效益**，隨著感應器與裝置的資料量不斷成長，讓邊緣裝置產生的原始數據量呈大幅增長。將所有數據傳輸至雲端不僅需要極高的頻寬，其帶來的傳輸、儲存與雲端處理，對於大規模部署而言，是極大的經濟負擔。所以讓邊緣人工智慧進行運算處理會比傳回雲端還具成本效益。

最後是**數據隱私與資料安全**，在醫療、金融和個人隱私保護嚴格的領域，將敏感數據上傳雲端處理存在重大的安全與法規風險。將資料保留在邊緣端，能確保數據在本地裝置內完成處理，降低數據安全和隱私權風險。

而邊緣人工智慧在實際落地中需要面對能源上及成本的限制，通常被稱作 PPA 指標 (Power - Performance - Area)，現代人工智慧的任務如複雜的視覺辨識或輕量級的生成式人工智慧對算力需求不斷攀升，但受限於體積和能源預算，邊緣裝置無法搭載雲端級別的硬體，因此如何在有限硬體資源下，運行越來越複雜且需保持一定精度的神經網路模型，成為邊緣人工智慧技術上的一項挑戰。

在深度學習中，常用的計算包括矩陣乘法與卷積乘法，這些計算涉及大量相同類型的運算，對於近似後的運算結果有一定的容忍度，可以依據使用者需求而捨棄一些精確度以換取更快的推論速度或更小的硬體成本。

在這次研究我們也根據此觀念，實作一個近似運算張量處理器，並額外設計一個硬體架構，提供資料補償值，進一步再增加準確度。希望透過新的補償硬體架構，在仍然比傳統張量處理器的硬體面積更小的情況下，達到更高的準確度。

1.3 文獻回顧

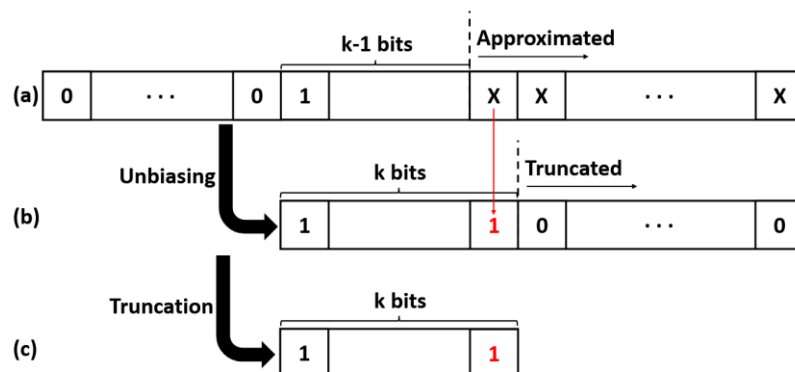
(1) Dynamic Range Unbiased Multiplier, DRUM [2] :

DRUM 是為了解決現有近似乘法器在可擴展性和誤差分佈上的限制所提出的。在訊號處理、電腦視覺和機器學習等容錯性應用中，乘法操作至關重要，但傳統精確乘法器的功耗和面積成本極高。而 DRUM 旨在透過犧牲可控的精確度來換取顯著功耗與面積的節省。其核心設計目標包括：

- (1) 動態性與可擴展性：設計能根據輸入操作數的實際數值動態選擇範圍，以維持準確性，並確保設計的優勢不會隨輸入位寬 n 的增加而衰減。
- (2) 無偏誤差分佈：確保乘法器引入的誤差分佈是無偏的（平均值接近於零），以避免誤差在重複計算中累積，進而提升推論準確度。
- (3) 可調配性：允許設計者通過單一參數 k （乘法核心位寬）來調整權衡準確度與功耗。

DRUM 的優勢在於其**動態範圍選擇**和**去偏置 (Unbiasing)** 機制。首先，透過最高有效位元檢測器 (Leading one Detector, LOD) 確定最高有效 '1' 所在的索引 t 。接著乘法器會基於 k 值，動態地選擇從索引 t 開始，向下連續 k 個位元作為近似輸入。而位於索引 $t-k+1$ 以下的位元將被截斷。

這種近似機制在於其無偏誤差分佈的實現。如果僅將截斷的低位元近似為 0，將會導致結果始終偏小，形成有偏 (biased) 的誤差。為了解決這個問題，DRUM 估計被截斷的 $t-k+2$ 個最低有效位元組成的數值期望值。假設操作數是均勻分佈的，其期望值約為 2^{t-k+1} ，如圖一所示。為了在硬體上近似這個期望值，DRUM 選擇將被截斷位元中最高位元設置為 1，而其餘更低的位元設置為 0。這種處理方式有效地將誤差分佈的平均值趨近零，使誤差呈高斯分佈。在實際應用中，這種無偏的特性極大地提高了準確度，因為正負誤差相互抵消，避免了誤差的累積，從而優於有偏乘法器。



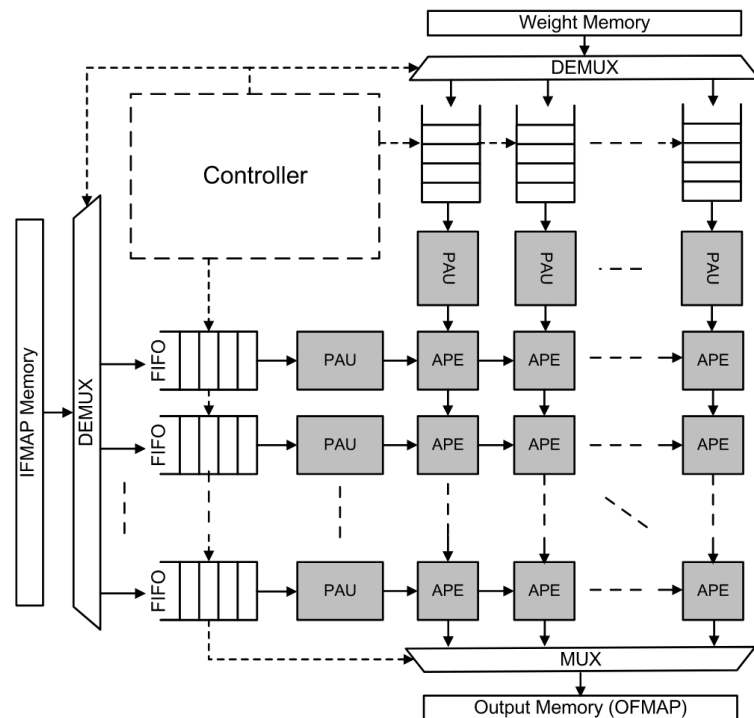
圖一：DRUM 的近似處理 [2]

(2) Approximate Computing Based Tensor Processing Unit, APTPU [3] :

APTPU 提出了一個基於近似計算的張量處理器架構，主要在克服傳統張量處理器在深度學習工作負載中面臨的高資源消耗與成本效益。APTPU 的主要創新在於將近似邏輯單元整合到脈動陣列中，並重新設計處理單元的結構，以新架構近似處理單元 (Approximate Processing Element, APE) 取代傳統乘加單元，由一個低精確度乘法器和一個近似加法器所組成，近似加法器採用了 LOA (Lower-part OR Adder) 來處理累積操作。除此之外，APTPU 最主要的架構優勢為預近似單元 (Pre-Approximate Unit, PAU)，預近似單元負責在數據進入近似處理單元之前，對高精確度資料 (如 32-bit) 進行動態截斷，轉換為近似處理單元所需的低精確度 (如 8-bit)，透過將複雜動態截斷邏輯移出近似處理單元，顯著降低了其關鍵路徑延遲 (critical path delay)，從而提高了整體時脈頻率和性能。

APTPU 的近似乘法設計繼承並應用了前面提到的概念，如 APTPU 在預近似單元中實現的動態截斷，其目的是定位操作最高有效位，並截取所需的 k 位元進行計算。採用 DRUM 的優勢在於其無偏誤差特性，在脈動陣列中，PE 會被重複使用，DRUM 的無偏誤差特性能使得正負誤差可以在多個計算週期中相互抵消並平均，從而維持複雜深度學習模型的準確度。

先前動態調整精確度的近似計算方法雖然有效，但動態調整邏輯本身會增加硬體複雜度和延遲，限制了操作頻率。因此下一篇論文透過預處理 (Pre-processing) 和固定化來簡化處理單元內部的結構。



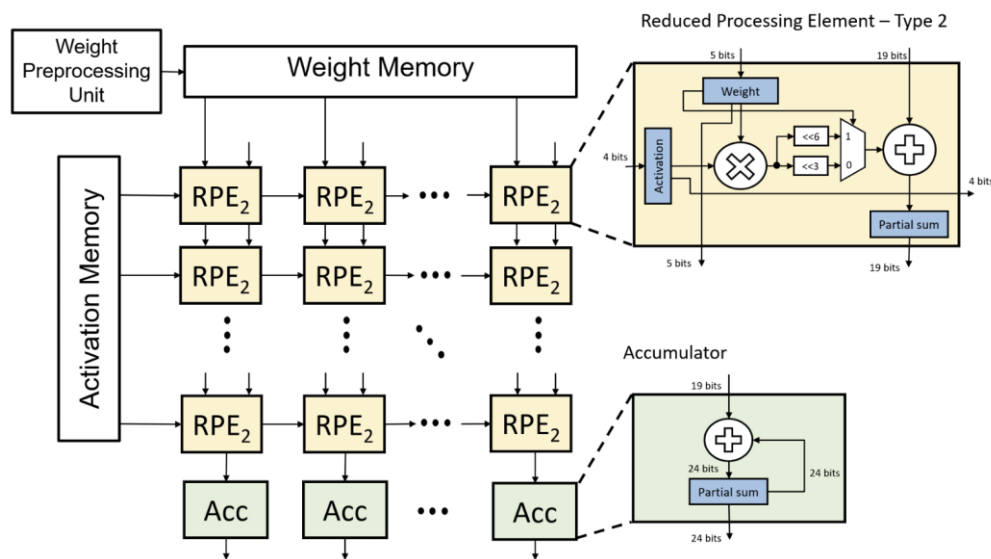
圖二：APTPU 架構圖 [3]

(3) Reduced-Precision Tensor Processing Unit [4] :

RPTPU 主要是透過權重的特性來針對張量處理器中的脈動陣列內部的處理單元，進行面積成本的優化，由於深度學習中訓練好的權重值普遍接近零，當這些權重值轉換成定點數 (Fixed-Point) 格式後，從高位元至低位元會產生連續的 0 或連續的 1，並將其稱作重要的連續位元 (Most-Significant Run, MSR)。

假設有一個權重值從高位元算起有四個重要的連續位元，也就是出現了 4 個連續的位元 0 或連續的位元 1，這時候就可以用一個符號位元 (Sign Bit) 來取代這四個連續位元，並且不會造成任何準確度的損失。所以可以透過這個重要連續位元的特性來使資料位寬 (Bit Width) 縮短，這樣在張量處理器中的脈動陣列裡，每個處理單元內部將可以用更小的乘加單元來進行計算，能使整體的面積與能量消耗減少，更短的位寬運算還能使乘法器更快速地計算輸出結果，以提升整體效能。

RPTPU 的設計，借鑒了前兩篇論文的一些概念，並加以改良。在 RPE Type-2 中，使用了 DRUM 無偏特性，以最高被截斷位元設為 1 作為期望值取代被捨去的位元，以補償縮減字長所失去的精確度。而對於 APTPU 的改良點則在於將類似預近似單元的硬體，權重預處理單元 (Weight Preprocessing Unit, WPU)，放置在權重記憶體前方，在權重存入記憶體之前，就先在權重預處理單元中將 MSR 替換成符號位元來縮減權重字長，並將複雜的精確度縮減和位移判斷固定化，PE 內部只需執行固定長度的乘法和簡單的位移判斷（而非計算位移次數的總和），降低硬體開銷及提升操作頻率，但 Type-2 RPTPU 為了進一步減少硬體成本，其激活值(Activation) 甚至被縮短至 5 位元精確度，導致模型彈性不足，容易在特殊情況下得到極端的推論準確度，反映了進行低位元量化所帶來的準確度問題。



圖三：Type-2 RPTPU 架構圖 [4]

1.4 專題貢獻

我們提出的張量處理器架構在模擬結果中展現出優良的準確度表現，證明了其通過權重位元分布壓縮與補償損失架構的有效性。我們分析了四個常見模型，並利用 NMIST 手寫辨識集來進行訓練，通過實驗表明了四種模型訓練後的權重皆有約 99 % 展示出 MSR-4 之特性，也代表了有約 99 % 的權重可以進行無損壓縮，而我們可以通過少量的補償陣列去補償約 1 % 損失，達到了在硬體成本下降的同時，依然可以維持高推論準確度。

根據使用 Pytorch 模擬不同模型下的準確度結果，可以觀察到在硬體成本較傳統張量處理器更少的狀況下，我們的架構在 MLP 和 LeNet 模型中的準確度優於傳統 INT8 量化方法。這項結果突顯了此架構在保持甚至超越傳統量化精度的同時，進一步實現硬體優化的能力。

另外，為評估架構硬體開銷，我們使用 Synopsys Design Compiler 對所提出的降低精確度處理單元 (Reduced-Precision Processing Element, RPE)、補償精確度處理單元 (Compensated-Precision Processing Element, CPE) 以及傳統處理單元進行了硬體合成比較分析。

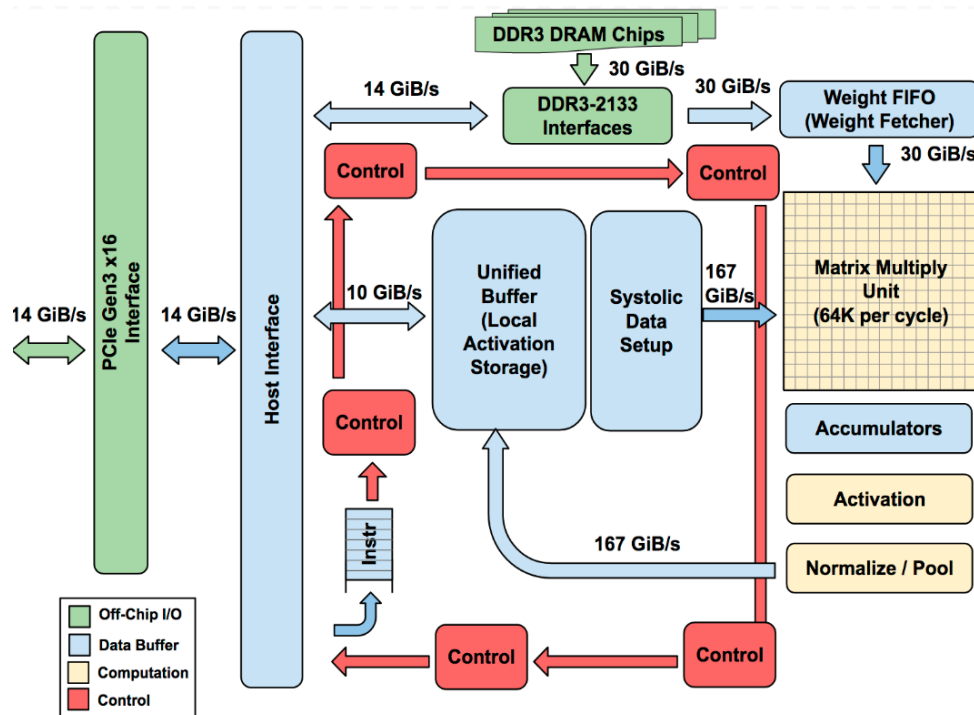
結果顯示，在一個 256×256 的脈動陣列中，即使額外加入 256×3 的補償陣列去維持準確度，整體脈動陣列硬體成本開銷相較於傳統 INT8 量化架構仍減少了 16.64%，並且由於權重壓縮至 5 位元、激活值壓縮至 7 位元，雖然加入了一個 256×3 的 4 位元補償記憶體，但其晶片上記憶體 (On-Chip Memory) 還是比傳統架構減少 18.46%。

此外，由於計算量的減少，功耗也顯著下降，同時帶來了性能的提升，其每秒兆次運算 (Trillions of Operations Per Second, TOPS) 比傳統架構快 6.54，實現了更少的硬體佔用、更低的能耗和更高的運算效率，並維持高精準度，充分展示了本專題在邊緣人工智慧運算加速器設計上的重要貢獻。

二、理論背景

2.1 張量處理器 [1]

Google 所提出的第一代張量處理器為專為深度神經網路推論設計的加速器，其架構核心如圖四所示。張量處理器以大規模整數矩陣運算陣列搭配高頻寬的資料通路構成完整的推論資料流，並透過規律的資料傳遞與可預測的延遲特性滿足資料中心對高吞吐量與低延遲的需求。如圖所示，Unified Buffer 運算過程中主要的激活值儲存空間，並以高頻寬路徑連接至 Systolic Data Setup，使資料能持續且穩定地送入矩陣乘法單元。同時，權重則由外部 DDR3 記憶體經由 Weight FIFO 以固定速度輸入，使激活值資料流與權重值資料流得以分離，提高資料供應的一致性並降低 DRAM 存取干擾。矩陣乘法單元採用脈動陣列，使資料在陣列內沿固定方向流動並累積至晶片上暫存器，進一步減少記憶體往返。運算結果經由 Accumulators、Activation 及 Normalize/Pool 等模組處理後重新寫回 Unified Buffer，形成高度封閉且高效的晶片上的運算循環。整體架構透過多條百 GB/s 級資料通道與簡化控制邏輯，展現出張量處理器在推論工作負載下同步兼具高效能與低延遲的設計特性



圖四：張量處理器架構圖 [1]

2.2 Data Flow (WS/OS) [5]

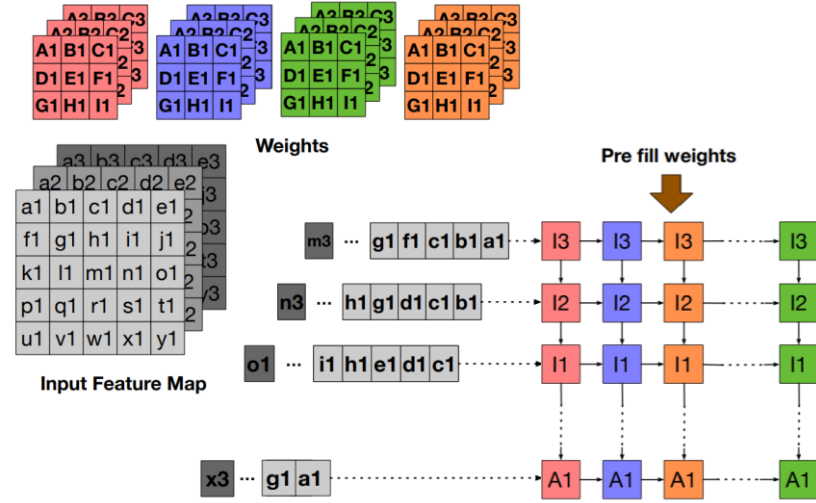
DNN 中最頻繁、也最耗能的運算是卷積層和全連接層中的乘積累加運算操作。這些運算基本上是由兩個巨大的矩陣，輸入激活矩陣與權重 (Weight) 矩陣的相乘。在傳統的中央處理器或圖形處理器架構中，數據的頻繁存取是主要的功耗瓶頸。特別是權重，它們的總數量往往非常龐大，而且在計算一個輸出特徵圖 (Output Feature Map, OFM) 的過程中，權重會被高度重複使用。如果每次計算都需要從晶片外主記憶體中重新載入這些權重，整個加速器的效率將會大打折扣，功耗也會變大許多。

而作為深度學習領域專為加速大規模矩陣乘法運算而設計的硬體架構，其效率的基礎之一便是其獨特的數據流設計，包含了權重固定 (Weight Stationary, WS) 及輸出固定 (Output Stationary, OS) 兩種資料流。

(1) 權重固定資料流：

其核心概念簡而言之就是將權重數據固定在靠近計算單元的位置，以最小化權重的移動，降低資料提取與傳輸功耗。具體而言，在張量處理器的脈動陣列中，每一個獨立的處理單元都會預先載入一個或一組特定的權重值，並將這些權重靜止不動地存放在處理單元內部的暫存器或局部記憶體中，持續整個計算階段。當權重被固定時，張量處理器便能讓輸入特徵圖 (Input Feature Map, IFM) 資料以流動的方式高效地流經整個脈動陣列，輸入特徵資料就能像流水線一樣，按照預定的週期從一個處理單元傳遞到下一個處理單元。當一筆資料流經處理單元時，它會與處理單元內部靜止的權重進行乘法運算，並將結果與先前累積的部分和 (Partial Sum) 相加。這個部分和隨後也會隨著輸入數據流向下一個處理單元，繼續進行累積。

這種設計帶來了幾個關鍵的優勢。首先，由於權重數據只需載入一次，就能在晶片上被多次重複使用，因此極大地減少了對晶片外高能耗記憶體頻寬的需求，從而大幅降低了功耗。其次，這種規律、同步的數據傳輸模式使得數據的移動和計算能夠完美對齊，避免了複雜的控制邏輯電路，使得張量處理器能夠以更高的時脈頻率和更簡潔的硬體設計運行。



圖五：權重固定資料流 [5]

在卷積層（Convolutional Layer）或全連接層（Fully Connected Layer）中，輸出特徵圖的每個元素都是由大量的輸入特徵資料與的乘積，經過一連串的累加而得。這個數據不斷增長的中間結果，稱為部分和。在傳統計算架構的效率瓶頸，往往出現在對這些部分和的頻繁存取上，每當一個新的乘積產生，部分和就需要從暫存器讀出、與新乘積相加，再寫回暫存器。在一個擁有數百萬甚至數十億次乘加運算的層中，其讀取和寫入操作所消耗的能量和時間十分巨大。

(2) 輸出固定資料流：

輸出固定資料流正是針對此瓶頸提出之解決方案，將這些高能耗存取的部分和數據固定在靠近計算發生的位置。在張量處理器的脈動陣列中，每個獨立的處理單元不再僅是一個執行乘加運算的邏輯電路，而是一個具備局部緩衝能力的累積站點。

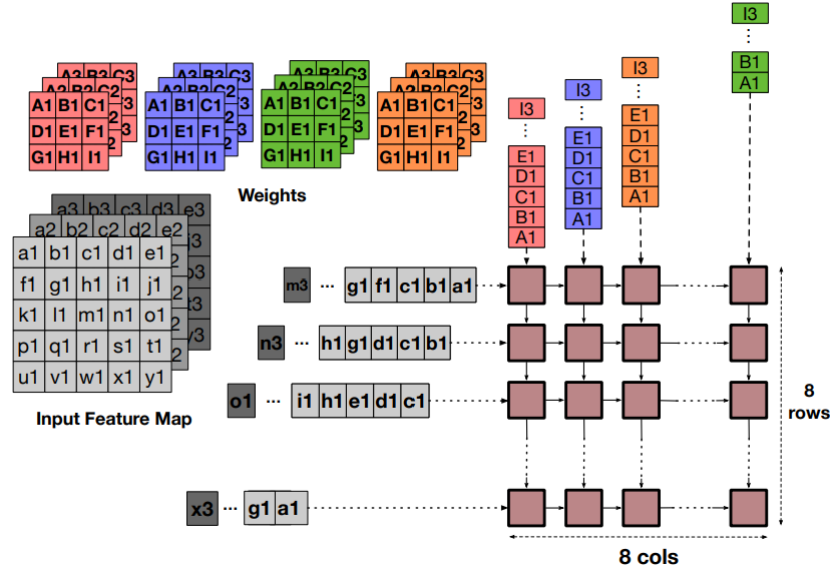
在一個輸出固定資料流的工作週期中，每個處理單元會被預先指定負責計算輸出矩陣 Y 中的一個或一組特定元素，一旦這個工作確定，該 PE 內部便會分配一個專用的暫存器來儲存該輸出元素的部分和。這個部分和在整個計算階段中都會保持靜止，直到所有的結果（即所有 $X * W$ 的乘積）都被加到它身上。

當乘加累積運算啟動時，輸入特徵資料和權重資料必須以精確同步的方式，從脈動陣列的邊緣被饋入。這兩種數據流以特定的方向（對角線方向）同步流動，穿越整個 PE 網格。當一個特定的輸入特徵元素 X 和一個特定的權重元素 W 在正確的時脈週期到達負責累積其結果的目標 PE 時，PE 執行乘法 $X * W$ ，隨後將結果加到其內部儲存的靜止部分和上，並立即更新該累積器的值。

當所有數據流動完成，且處理單元內的累積運算已結束，PE 便會

將最終的輸出結果釋放至下一級的緩衝區或片外記憶體。隨後該處理單元的累積器將會被重置，準備計算下一組輸出元素資料。

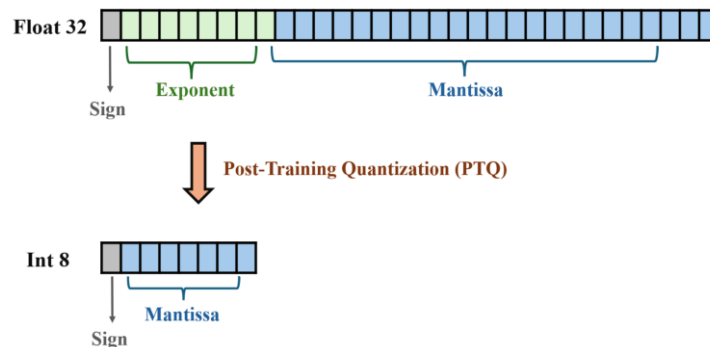
通過將部分和固定在處理單元內，每次累積都成為一個純粹的計算操作，消除了對中間結果的讀取和寫回操作所涉及之記憶體存取能耗。



圖六：輸出固定資料流 [5]

2.3 訓練後量化 (Post-Training Quantization, PTQ)

訓練後量化是一種高效的神經網絡模型壓縮方法，旨在將預訓練好的全精度（如 FP32）模型轉換為低精度表示（如 INT8），如圖七所示，無需依賴完整的訓練流程或大量的計算資源。正因訓練後量化具備快速、低資源依賴的特性，使其完美契合專注於推論加速且具備嚴苛硬體成本與功耗預算的邊緣運算環境。但同時訓練後量化的技術也面臨著精確度不夠的問題，如何在有限的硬體資源與可接受的推論準確度之間找到最佳平衡點，是邊緣運算部署與模型量化領域中的一大難題。



圖七：訓練後量化示意圖

2.4 Most Significant Runs (MSRs) [6]

連續重要位元 (Most Significant Runs, MSRs) 是深度神經網絡硬體加速器優化中一項關鍵的觀察結果，它利用了訓練完成後權重值的固有統計特性來實現數據精確度的無損或近似無損縮減。這個概念的提出，是基於對深度神經網絡模型中大量的權重資料進行定點數格式的分佈觀察。

當一個深度神經網絡模型經過訓練收斂後，大多數權重值會聚集在零附近。無論這些權重是正數還是負數，當它們被轉換成二進制的定點數格式時，從最高有效位元 (MSB) 開始向低位元觀察，會出現一長串連續的相同位元值，這些連續的相同位元就是所謂的「連續重要位元」。

0.004096	-0.420437	0.153534		<u>00000001</u>	<u>11001010</u>	<u>00010100</u>
0.114851	0.036513	-0.028751		<u>11110001</u>	<u>00000101</u>	<u>11111100</u>
0.174329	-0.028819	0.143463	➡	<u>00010110</u>	<u>11111100</u>	<u>00010010</u>
0.243178	0.036531	-0.082088		<u>11100001</u>	<u>00000101</u>	<u>11110101</u>
0.025325	-0.313233	0.173153		<u>00000011</u>	<u>11010111</u>	<u>00010110</u>
Original Data				Fixed Point		

圖八：MSR 示意圖

這連續的 '0' 或 '1' 在數值上並不包含多少額外的資訊。例如，在一個 8 位元的定點數中，00001101 的前四個 0 都是冗餘的，因為第一個非零位元已經確定了數值的規模，因此這四個 0 僅僅只代表資料的正負數。而 MSRs 則被定義為從 MSB 開始，到第一個與符號位元不同的位元出現為止的連續相同位元序列。

2.5 期望值補償 [2]

當深度神經網絡的權重值從訓練時常使用的 32 位元浮點數，被量化至推論時的 16 位元定點數時，雖然已經能大幅節省硬體資源，但若想更進一步追求硬體成本優化，如將運算精確度再下探至 8 位元，則會面臨推論準確度的大幅損失，這種準確度的損失源於對數據低位元的捨棄，亦即傳統的截斷，舉例來說，從 16 位元到 8 位元的精確度縮減，意味著最低的 8 個位元將被無條件捨棄，而這些被捨棄的數值，是造成模型準確度下降的原因。這種捨棄導致的誤差是有偏的，因為誤差總是指向負值（近似值總小於或等於真實值），在大量的乘積累積運算中，這些有偏誤差將不斷累積，最終導致推論結果與原始高精度模型產生顯著偏差。

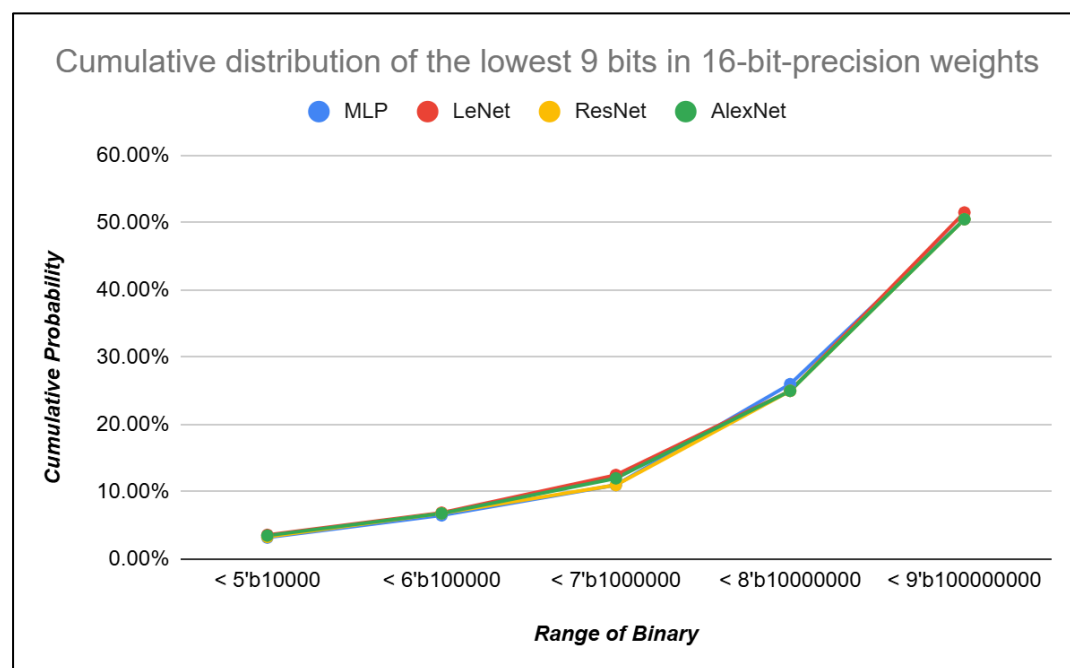
為了在實現低成本硬體的同時，有效解決這種有偏誤差累積的問題，我們必須使用一種更精確的近似方法，這部分需要對被捨棄的數據進行深入的統計分析。因此，我們將聚焦 16 位元精確度權重值中最低的 9 個位元。通

過分析這部分數據的分佈特性，我們尋求建立一個近似值，用於補償捨棄低位元所造成的數值損失，從而使 8 位元精度的權重能夠在數值上更接近 16 位元精度，以有效地降低誤差。

我們對累積分佈函數 (CDF) 中最低 9 個低位元組成的數值進行了統計觀察。累積分佈函數曲線明白地揭示了這些低位元數值的累積分佈情形，如圖九所示，我們可以觀察到一個關鍵現象：在 MLP、LeNet、ResNet 和 AlexNet 這幾種神經網絡模型中，約 50% 的權重值，其最低 9 位元組成的數值都小於二進制數 100000000，而另一半則大於這個數值。

這個 100000000 二進制數值（第 9 個位元為 1，其餘 8 個位元為 0）暗示著其在機率平均分佈的情況下，所有權重值被捨棄部分的期望值 (Expected Value) 大約會等於這個數值，即 100000000。也就是說，如果我們在 16 位元權重值中，將最低的 8 個位元全部捨棄，並將第 9 個位元設定為 0，那麼產生的誤差會是有偏的。然而，如果我們將第 9 個位元設置為 1，這個操作在就相當於我們在用期望值來近似被捨棄的數值。

總而言之，若我們將 8 位元精確度的 LSB 設為 1 作為期望值近似補償，就能有效地將原本 16 位元權重值有效地轉換成了數值上等價於 16 位元精度、但硬體位寬僅為 8 位元的近似權重值。



圖九：精確度 16 位元權重中最低 9 位元的資料累積分佈

三、實驗數據

3.1 實驗模型介紹

本篇論文使用 Pytorch 深度學習框架為平台來模擬實驗結果，並使用 MNIST 手寫辨識資料集作為訓練與推論資料集，使用的深度神經網路模型包括：

(1) MLP [7]：

此深度神經網路架構由三層全連接神經網路構成。輸入層包含 784 個神經元，輸出層包含 10 個神經元。訓練完的準確度為 98.08%。

(2) LeNet [8]：

此模型由五層神經網路構成，是早期卷積神經網路 (CNN) 的經典架構。前三層為卷積層，後兩層為全連接層，訓練完的準確度為 98.05 %。

(3) ResNet [9]：

此模型由十八層神經網路架構構成，包含十六層卷積層與一層全連接層。另有一層預處理卷積層(不計入總層數)，訓練完的準確度為 99.61%。

(4) AlexNet [10]：

此模型由八層神經網路架構構成，包含五層卷積層與三層全連接層。此模型是深度學習領域的重要里程碑，對圖像辨識的發展具有重大意義，訓練完準確度為 99.56%。

以下表一為各模型架構與實驗結果，包含層數 (Layer)、資料集 (Dataset)、輸入尺寸 (Input Dimensions)、輸出分類 (Output Classes) 和訓練的準確度 (Train Accuracy)，其中層數包含卷積神經網路層數 (CONV) 與全連結神經網路層數 (FC)。

而表二為各模型訓練的超參數設定，包含優化器 (Optimizer)、學習率 (Learning Rate)、學習率調整器 (Learning Rate Scheduler)、正規化 (Regularization)、訓練次數與批次大小 (Epochs / Batch) 和損失函數 (Loss Function) 的設定。

表一：各模型架構與實驗結果

Model	MLP	LeNet	ResNet	AlexNet
Layers (CONV/FC)	3 (0/3)	5 (2/3)	17 (16/1)	8 (5/3)
Dataset	MNIST	MNIST	MNIST	MNIST
Input Dimensions	28x28	28x28	28x28	28x28
Output Classes	10	10	10	10
Test Accuracy	98.08%	98.05%	99.61%	99.56%

表二：各模型訓練超參數設定

Model	MLP	LeNet	ResNet	AlexNet
Optimizer	Adam	Adam	Adam	Adam
Learning Rate	0.0001	0.000055	0.001	0.001
LR Scheduler (step / gamma)	N/A	N/A	7 / 0.1	7 / 0.1
Regularization	N/A	N/A	L2 ($\lambda=1e-4$)	L2 ($\lambda=1e-4$)
Epochs / Batch	10 / 64	10 / 64	15 / 64	15 / 64
Loss Function	Cross Entropy Loss			

3.2 權重 MSR 分布分析

在 2.2 小節中，我們介紹了在邊緣運算或是需低成本的推論環境中，目前主流的作法是採用訓練後量化 (Post-Training Quantization, PTQ)，PTQ 原來會需要 32-bit 或 16-bit 的浮點數量化為 8-bit 的定點數下去做推論。由於訓練完的權重分布大多集中在零附近，因此大部分的權重會出現我們在 2.3 節所提到 MSR 特性。因此，我們根據上述四種模型進行 MSR-N 的分布分析，如下表三所示，可以發現如果我們對 MSR-4 進行壓縮的話，將其權重壓縮成 5-bit (其中 1-bit 表示正負，4-bit 表示權重數值大小)，其帶來的準確度損失僅約 1%。

表三：各模型權重之 MSR-N 分布分析

Model	MLP	LeNet	ResNet	AlexNet
MSR-3	99.99%	99.98%	99.99%	99.99%
MSR-4	99.98%	98.90%	99.61%	99.98%
MSR-5	98.0%	88.13%	99.25%	99.37%
MSR-6	78.23%	53.41%	99.12%	97.38%
MSR-7	40.42%	27.35%	85.52%	84.31%

此外，我們觀察到一個現象：相較於 LeNet，ResNet 和 AlexNet 等更複雜的模型中，儘管訓練參數多出許多，但其訓練後的權重 MSR-4 分布占比卻提高不少。這其實是因為在這些複雜模型中，通常會使用 L2 正規化 (L2 Regularization) 的方法，去避免出現過擬合 (Overfitting) 的情況，L2 正規化透過在損失函數中加入權重的平方項，促使權重值趨近於零，進而提升了 MSR-4 權重的比例。

此結果表明，利用權重 MSR 分布特性進行壓縮的可行性並不受模型複雜度而有所限制，驗證了這種量化方法在不同模型下的通用性，也對複雜、高精度的深度學習模型部署到資源受限的邊緣運算裝置上，提供了更具潛力的高效率量化路徑。

雖然利用 MSR-4 僅會帶來 1% 的損失，但其仍然會導致準確度有所下降，如果我們可以通過少量的補償架構來補償這 1% 的損失，將有助於在降低硬體成本的同時，維持的推論準確度。我們以 Google 所提出的張量處理器架構為例，其包含了 256 x 256 的脈動陣列，如表四所示，最差的情況每 256 個權重中會有 2.9 個權重沒有 MSR-4，因此，我們只需要設計一個 256 x 3 的補償脈動陣列，用來補償準確度損失，即可維持推論準確度。

表四：各模型 Non-MSR-4 / 256 分析

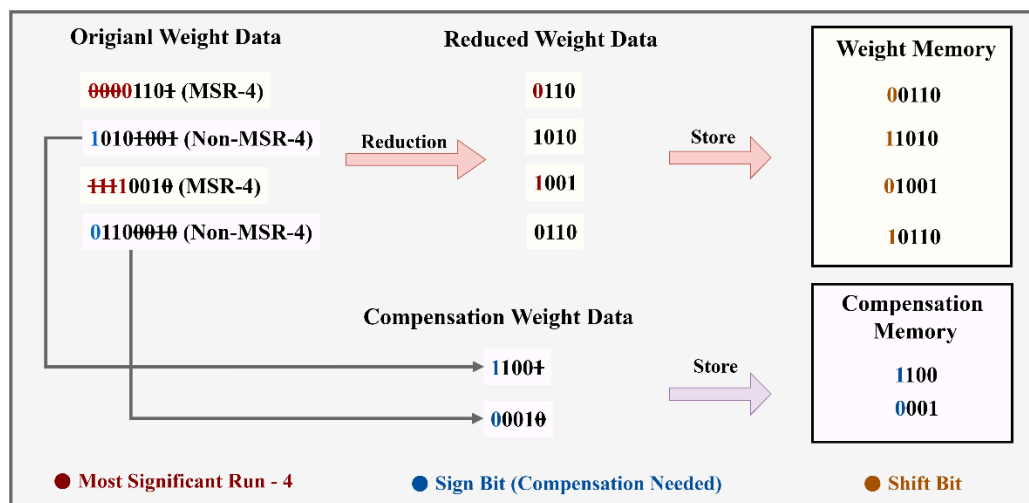
MSR-4 % & Non-MSR-4 / 256 (After Training)				
Model	MLP	LeNet	ResNet	AlexNet
MSR-4 %	99.98%	98.90%	99.61%	99.98%
Non-MSR-4 / 256	0.1	2.9	0.1	0.1

3.3 權重儲存與補償方式

如圖十所示，我們將權重資料經過權重預處理單元 (Weight Pre-processing Unit, WPU)，權重預處理單元的功能是判斷權重資料其是否屬於 MSR-4，並將所有權重進行截斷 (Truncation) 後送入權重記憶體 (Weight Memory)。比較特別的是，對於沒有 MSR-4 的權重資料，其截斷後的部分會被送入補償記憶體 (Compensation Memory) 中儲存。

根據 2.4 節所述，我們採用期望值補償的策略來補償 PTQ 所帶來的準確度損失，因此所有權重的最低位元 (LSB) 皆不需存入記憶體中，只需要在運算的時候，把這個 1-bit 的 1 做為最低有效位元即可，減少了一些記憶體的儲存空間。

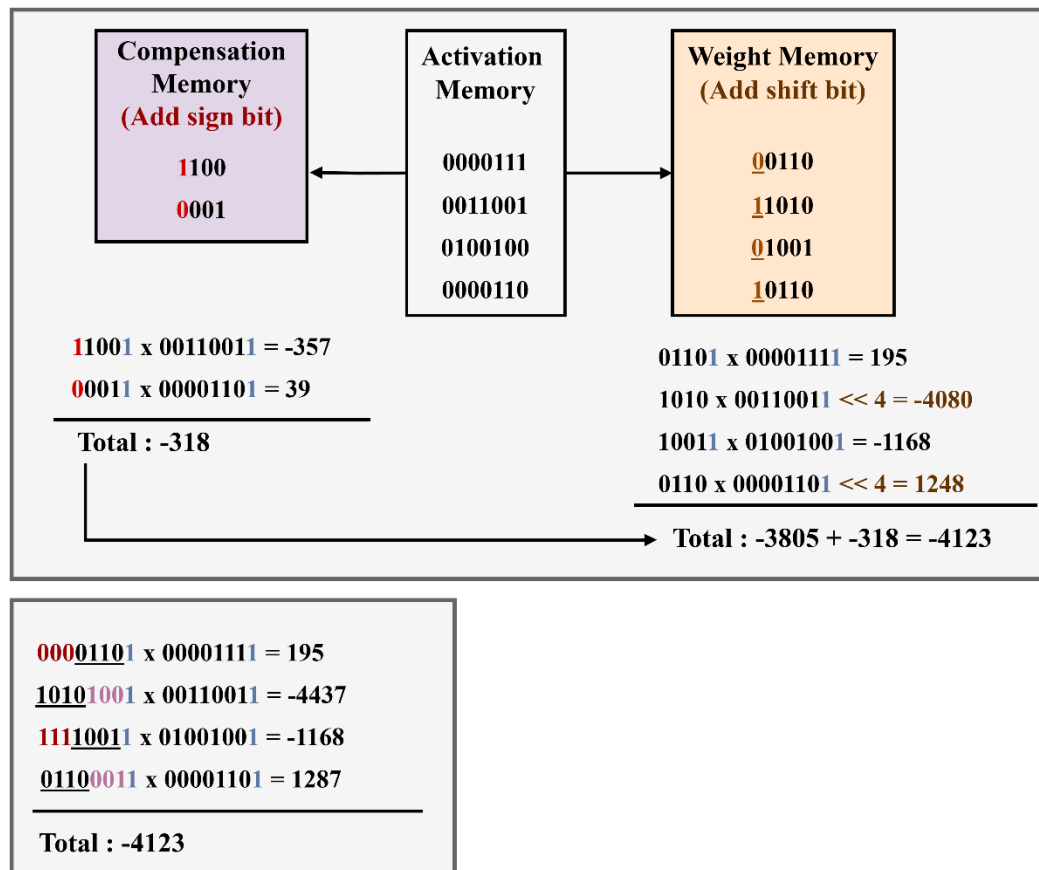
對於權重記憶體的儲存格式，我們需要增加一個位移位元 (Shift-bit)，用以標識該權重是否為 MSR-4。如果是 MSR-4 的話，其位移位元為 0，因為其不需要做位移，而對於 Non-MSR-4 的權重來說，其位移位元為 1，因為其需要做位移才可以保持資料一致性，以避免後續計算出現錯誤。此外，在補償記憶體中，我們也需要再加入一個符號位元 (Sign-bit)，以確保補償值的符號與原權重值一致，避免在做乘加運算時出現錯誤。



圖十：權重資料儲存方式

圖十一展示了本研究中乘加運算與其補償機制的流程。對於 MSR-4 權重，我們首先加入最低有效位元的期望補償值，再與激活值進行乘法運算。同樣的，激活值也會進行最低有效位元的期望補償，以提升整體運算精度。然而，對於 Non-MSR-4 權重，我們不會直接加上期望補償值。這是因為其最低有效位元實際上包含在補償記憶體中所儲存的截斷部分，因此只需對補償資料進行期望值補償即可。此外，在處理負數時，系統會對資料取二的補數進行運算；但由於 Non-MSR-4 權重的最低有效位元位於補償精度脈動陣列 (Compensated-Precision Systolic Array) 中，所以當縮減精度脈動陣列 (Reduced-Precision Systolic Array) 遇到負數的 Non-MSR-4 權重時，需改以取一的補數來計算。

Non-MSR-4 權重在乘法運算後，會再向左位移四個位元，以恢復其原始數值，確保資料對齊與一致性。最終，將補償陣列與縮減陣列的運算結果相加，即可獲得無損失的乘加運算結果，從圖十一可以看出採用 MSR-4 壓縮與補償運算後的結果與原始運算結果一致。



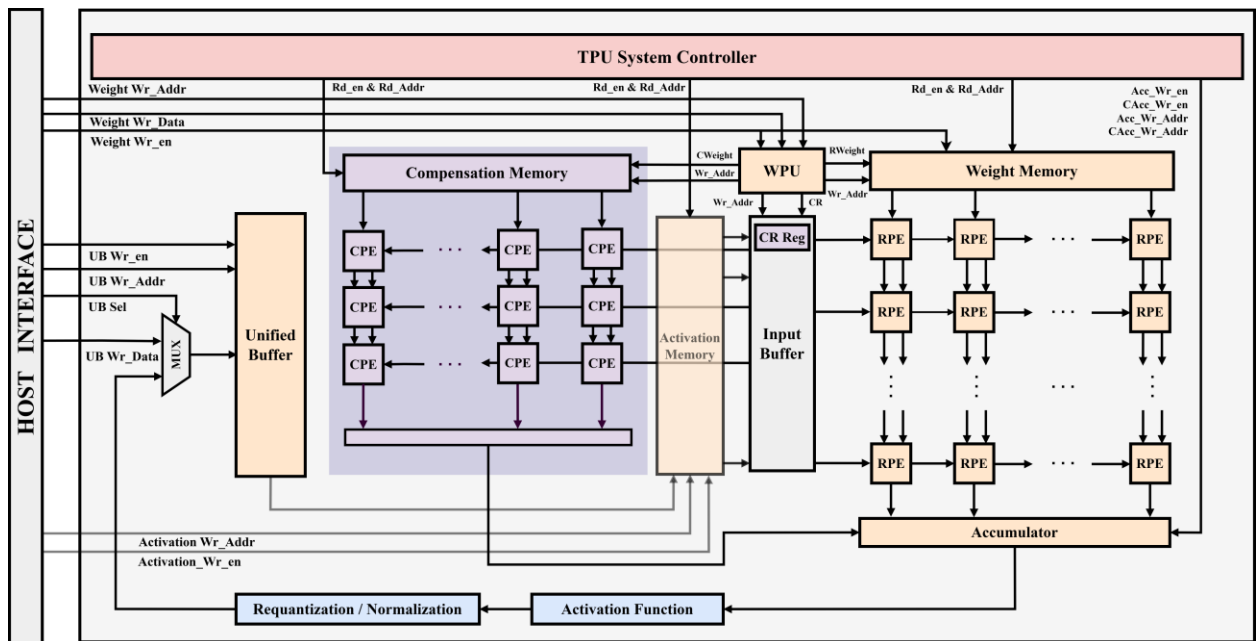
圖十一：乘加運算與其補償機制

3.4 所提出之張量處理器架構

圖十二為本研究所提出之張量處理器架構，其包含了權重預處理單元 (Weight Preprocessing Unit)、權重記憶體 (Weight Memory)、激活值記憶體 (Activation Memory)、補償記憶體 (Compensation Memory)、輸入緩衝器 (Input Buffer)、統一緩衝器 (Unified Buffer)、縮減精度處理單元 (Reduced-Precision Processing Element, RPE)、補償精度處理單元 (Compensated-Precision Processing Element, CPE)、累加器 (Accumulator)、激活函數 (Activation Function)、重新量化或正規化 (Re-quantization / Normalization) 以及整個張量處理器系統的控制器 (TPU System Controller)。

在此架構中，權重資料會先經由權重處理單元進行預先處理，隨後寫入權重記憶體與補償記憶體中。當運算開始時，張量處理器系統控制器依據第 2.2 節所述的權重固定資料流，將權重分配至 RPE 與 CPE 中。接著，激活值會由激活值記憶體寫入輸入緩衝器，並經由 45 度流水線 Pipeline 同步送入主體陣列與補償陣列進行運算。當運算完成後，結果會在累加器中進行補償，再經過激活函數與重新量化處理後，寫回統一緩衝器。隨後系統載入下一層的輸入資料，重複上述流程，依序完成整個深度神經網路的運算任務。

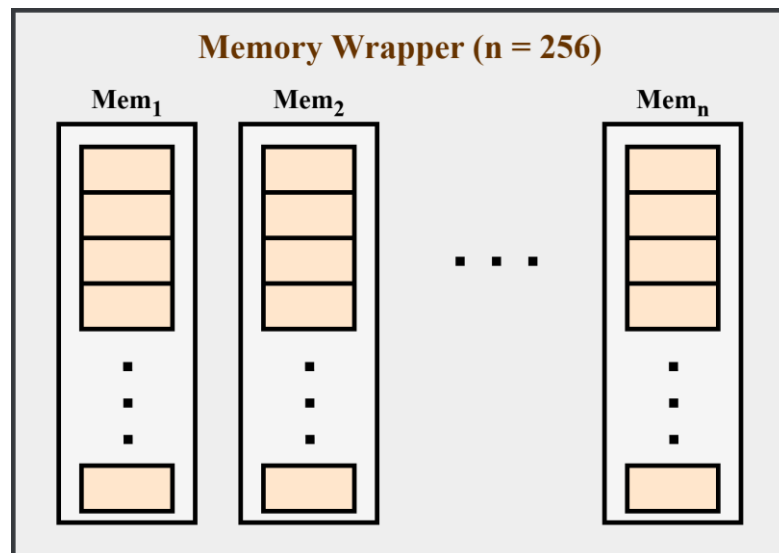
圖十二：本研究所提出之 TPU 架構



3.4.1 Memory Wrapper

在脈動陣列的架構中，以 256×256 為例，運算時需同時從權重記憶體與激活值記憶體各讀取 256 筆資料，以輸入至陣列中進行平行運算。然而，實際硬體中的單一記憶體通常僅能於一個時脈週期內讀出一至兩筆資料，導致無法滿足脈動陣列高並行度的資料吞吐需求。

為克服此一限制，本研究將原本單一的大型記憶體分割為 256 個獨立的子記憶體，並以 Memory Wrapper 的方式進行統一包裝與控制，如圖十三所示。當資料寫入時，Memory Wrapper 會根據地址與控制邏輯決定目標子記憶體，確保資料能被正確分配並儲存於對應的位置。透過此設計，系統能在同一個時脈週期內同時從 256 個子記憶體中讀出資料，成功解決傳統單一記憶體無法支援高並行的資料讀取問題，從而提升整體脈動陣列的運算效率與記憶體頻寬利用率。

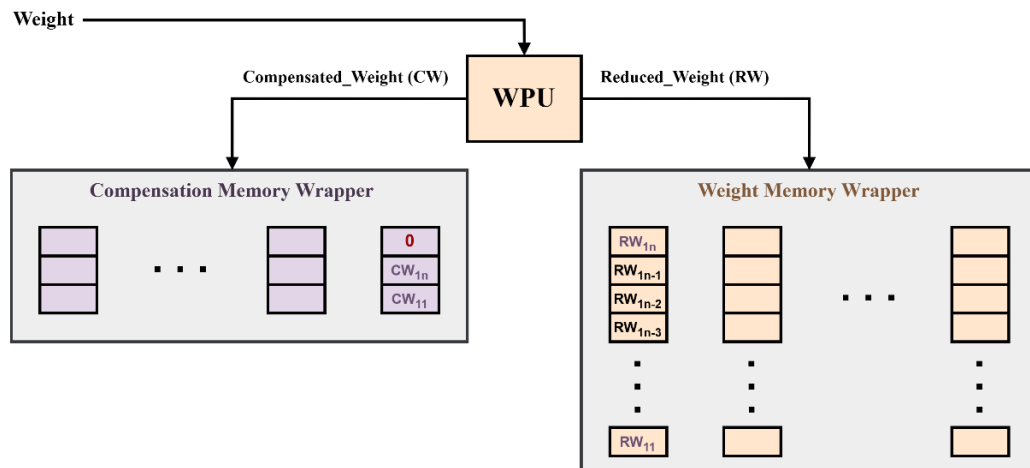


圖十三：本研究所採用的 Memory Wrapper 架構圖

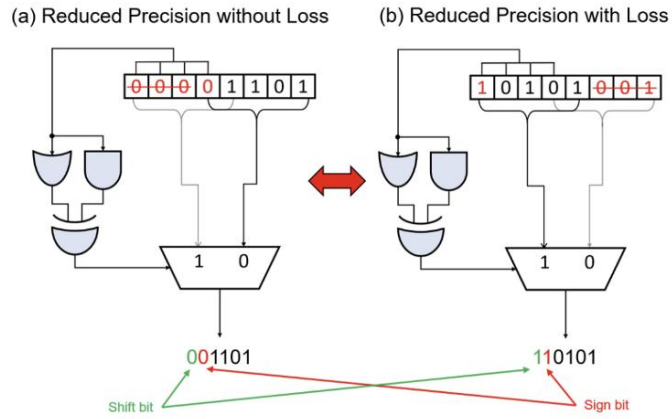
3.4.2 權重預處理單元 (WPU)

權重預處理單元 (WPU) 與權重及補償記憶體的資料路徑 (Datapath) 如圖十四所示。當權重預處理單元接收到權重資料時，會先利用圖十五的架構判斷其是否屬於 MSR-4 權重 [3]。若為 MSR-4 權重，權重預處理單元會依照第 3.3 節所述的方法將其截斷後寫入權重記憶體中；若非 MSR-4 權重，則同樣依第 3.3 節的方法將 Non-MSR-4 權重截斷並寫入權重記憶體，同時將被截斷的部分連同原始權重的符號位元一併送入補償記憶體。以圖十四為例，第 1 個與第 n 個權重 ($n = 256$) 為 Non-MSR-4 權重，因此可知第一個行 (Column) 中僅有兩個需要補償的值。為避免下一行的補償權重誤寫入補償記憶體的第一行，權重預處理單元會傳送 Change_col 訊號，通知補償記憶體進行換行操作。

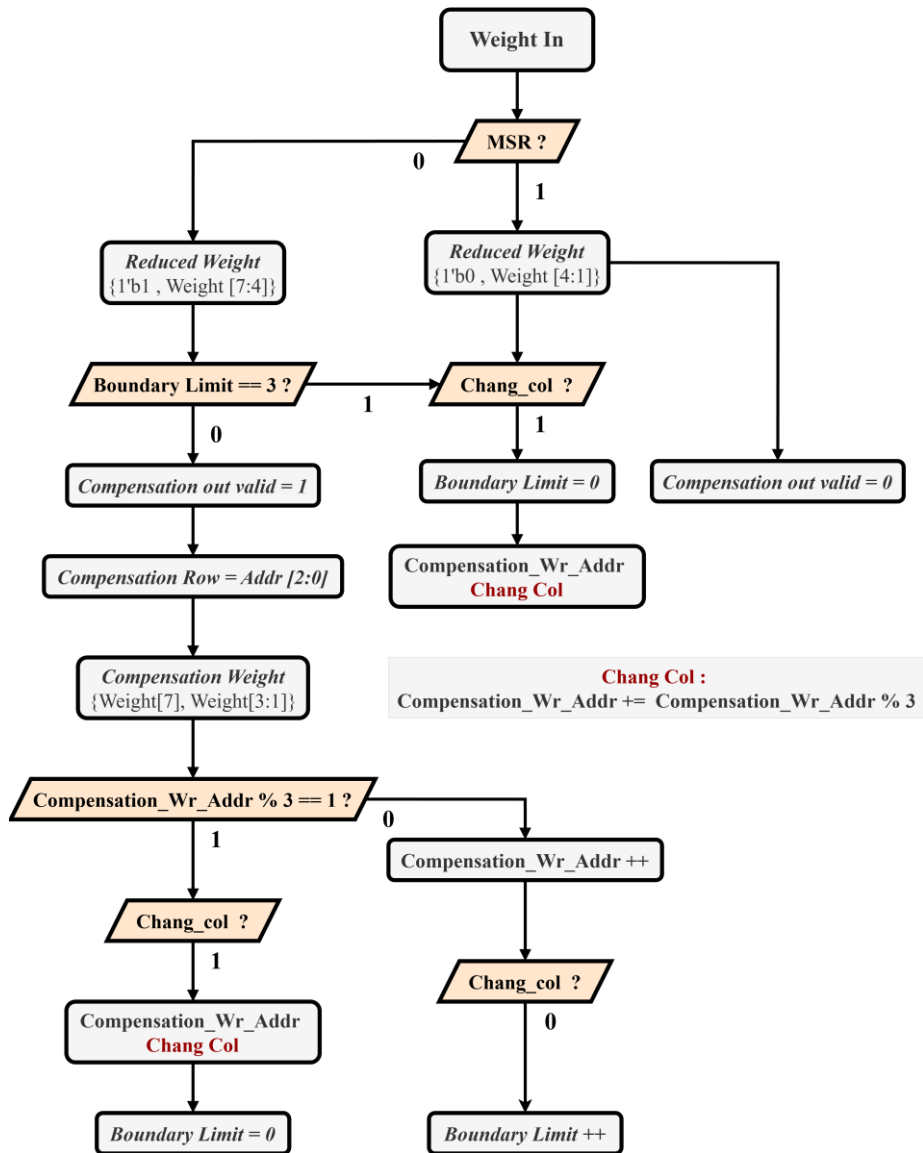
此外，因補償值所對應的列 (Row) 必須與相同的激活值相乘 (例如圖十四中的 CW_{11} 與 RW_{11})，因此權重預處理單元會將該補償值對應的列索引 (Row Index) 寫入輸入緩衝器的列索引暫存器 (Row Index Register)，以確保後續能透過輸入緩衝器將正確的激活值分配至補償陣列進行運算。值得注意的是，若在 256 個權重中出現超過 3 個 Non-MSR-4 權重，超出的部分將被忽略，不會儲存至補償記憶體中，其權重預處理單元運作流程圖如圖十六所示。



圖十四：本研究所提出之權重預處理單元與記憶體的資料路徑圖



圖十五：權重預處理單元判別與截斷架構圖 [3]



圖十六：權重預處理單元流程圖

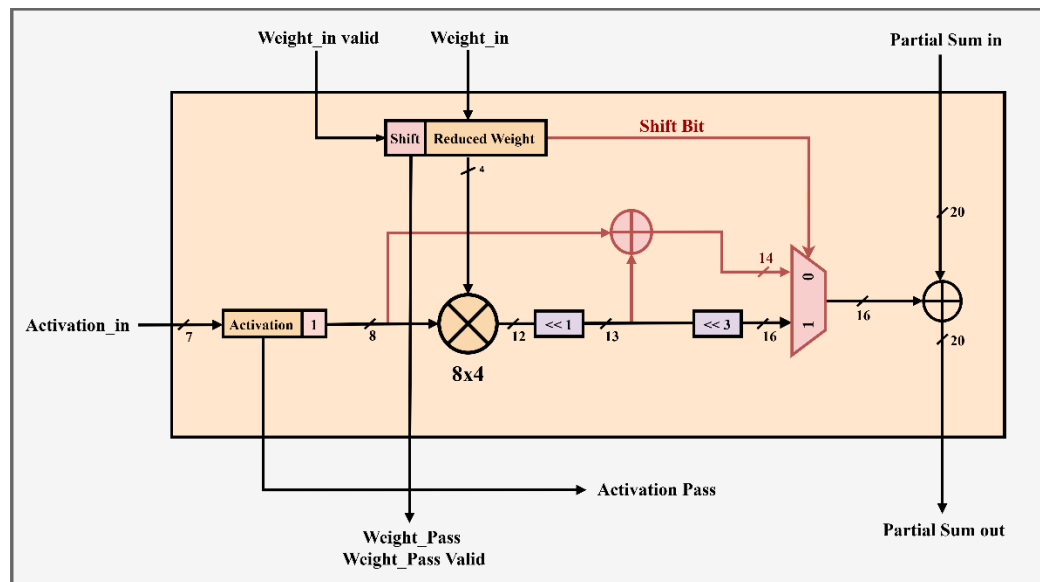
3.4.3 縮減精度處理單元 (RPE)

圖十七為本研究所提出之縮減精度處理單元 (RPE) 架構圖。RPE 的核心設計目標在於高效處理 MSR-4 與 Non-MSR-4 兩類量化權重，同時維持運算的精確度。由於 Non-MSR-4 權重的補償邏輯無法在此運算單元中直接進行最低位元期望值補償，因此本研究將 8×5 二的補數乘法器，改為由一個 8×4 的二的補數乘法器與位移暫存器、二的補數加法器所組成的架構。

具體而言，輸入的 4 位元縮減後權重與 8 位元激活值在 8×4 乘法器中進行乘法運算，產生 12 位元的乘積。接著，該乘積經由位移暫存器處理後，再加上一個激活值，便可實現 MSR-4 權重的最低位元補償，即達成 $W \times A + 1 \times A$ 的運算形式。

至於 Non-MSR-4 權重的運算結果，則會經過總共四次左位移，以與原始資料對齊，確保資料格式與數值的一致性，並且不會受到最低位元期望值補償。上述兩種運算結果最終皆會導入多工器，並由位移位元訊號決定最終輸出的資料路徑。

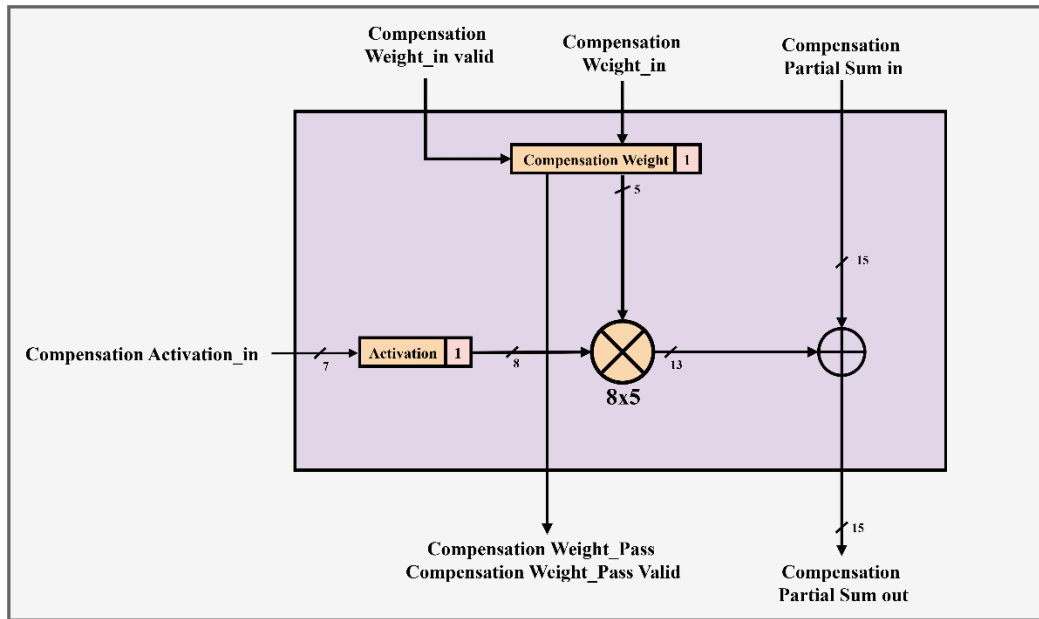
此外，本研究採用固定權重資料流的架構設計，因此需在運算前預先將權重載入至 RPE 的權重暫存器中。當 Weight_in valid 信號為 1 時，表示可進行權重載入；反之，則暫停載入操作。



圖十七：本研究所提出之 RPE 架構圖

3.4.4 補償精度處理單元 (CPE)

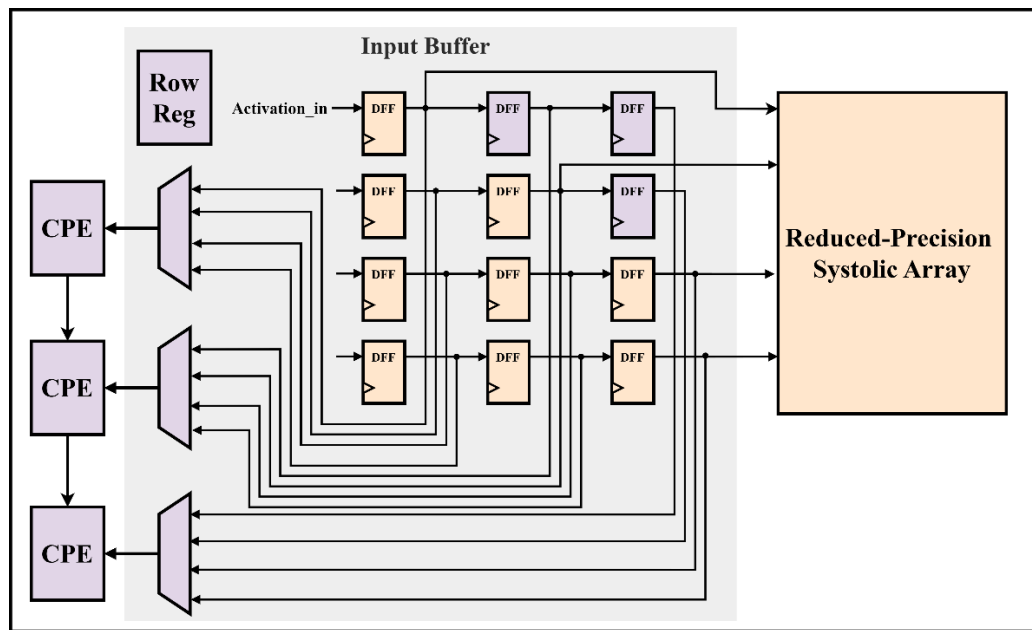
圖十八為本研究所提出之 CPE 架構圖，在這個處理單元中，我們會直接將補償權重的最低有效位元設為 1 做期望值補償，用來計算 Non-MSR-4 被截斷的部分。這個單元內含有一個 8x5 的二的補數乘法器與一個加法器，計算完後將其透過累加器進行補償運算。此外，由於權重固定資料流需要預先載入權重，因此這裡也需要 Compensation Weight_in valid 訊號來告訴 CPE 是否載入補償權重。



圖十八：本研究所提出之 CPE 架構圖

3.4.5 輸入緩衝器 (Input Buffer)

圖十九為輸入緩衝器 (Input Buffer) 架構圖，原先的輸入緩衝器僅為圖中橘色區塊，負責將激活值做正 45 度角的 Pipeline 後送到脈動陣列中。而在本次架構中，由於需要做補償，新增了紫色區塊的列指標暫存器與多工器，權重預處理單元會將 Non-MSR-4 權重的列指標送到列索引暫存器中存儲，接著多工器藉由對應的列索引選擇該 CPE 需要的激活值是哪一系列的。此外，在補償陣列中我們也需要做正 45 度角的 Pipeline，因此，第一個跟第二個列都需要 3 個暫存器，以供選擇。



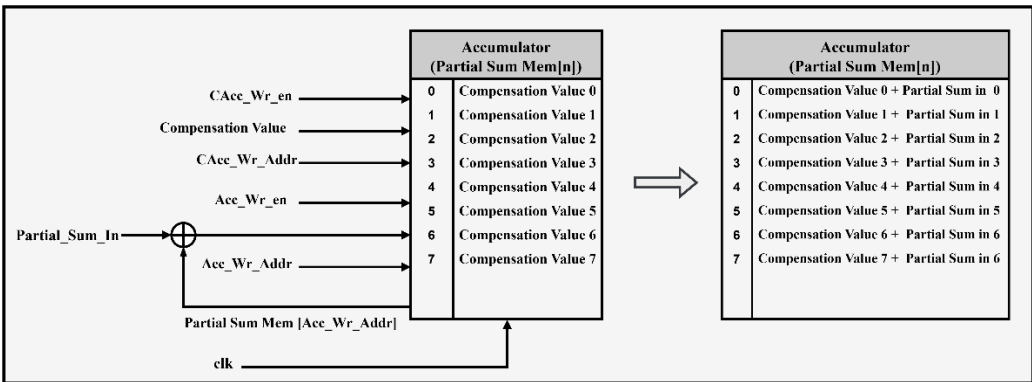
圖十九：本研究所提出之輸入緩衝器架構圖

3.4.6 累加器 (Accumulator)

圖二十為本研究所提出之張量處理器架構中的累加器 (Accumulator)。累加器除了負責儲存各處理單元的乘加運算結果外，當脈動陣列的規模小於輸入特徵圖 (Feature Map) 時，也可透過累加器將多次運算結果進行加總，以獲得完整的輸出結果。

此外，由於本架構包含補償陣列，且補償陣列每列僅需 3 個 CPE，相較於主體陣列每列 256 個 RPE，其運算速度明顯較快。因此，我們可先將補償陣列的運算結果預先寫入累加器中，待主體陣列完成運算後，再將補償值從累加器記憶體中讀出，與主體陣列的結果相加後寫回累加器，即可完成補償運算，確保整體結果無精度損失。此設計的優點在於無需為補償陣列額外配置獨立的累加器，透過讓主體陣列與補償陣列共用同一累加器，不僅降低硬體成本，亦提升了整體架構的資源利用效率。

綜上所述，本研究中累加器的實現亦採用第 3.4.1 節所述之 Memory Wrapper 架構，透過包裝多個記憶體，支援同時寫入與同時讀出多筆資料的操作，實現脈動陣列的高並行度。

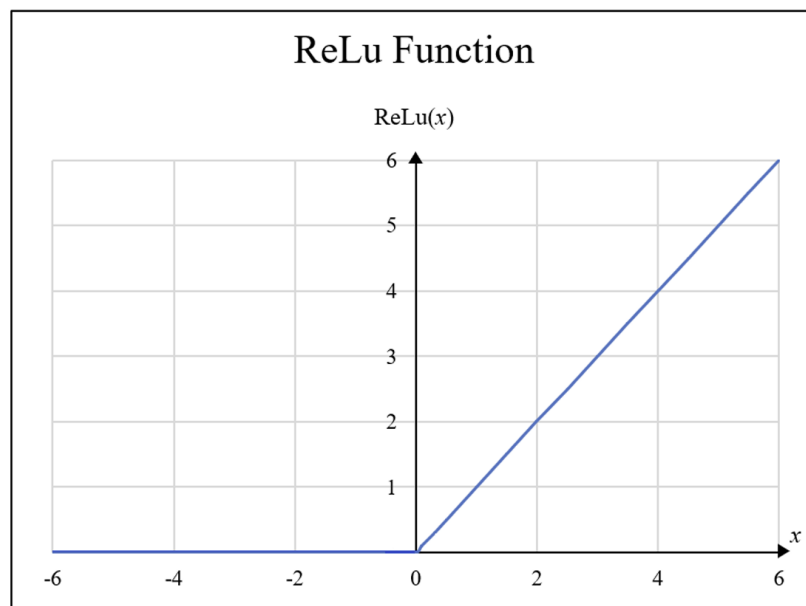


圖二十：本研究所提出之累加器架構

3.4.7 激活函數 (Activation Function)

在本次研究的實作中，採用線性整流單元 (Rectified Linear Unit, ReLU) 來作為激活函數。如圖二十一所示，在輸入值大於 0 時，輸出等於輸入；而當輸入值小於或等於 0 時，輸出則為 0。由於其運算形式簡單，具有極高的計算效率，特別適用於大型神經網路的訓練與推論過程。

此外，線性整流單元有效緩解傳統激活函數中常見的梯度消失或梯度過大問題，並具備良好的數值穩定性與收斂速度。因此，線性整流單元已成為近年深度學習模型中最廣泛使用的激活函數之一。



圖二十一：線性整流單元函數

3.4.8 重新量化 (Re-quantization / Normalization)

由於資料在經過乘加運算及累加器的累加與補償運算後，其結果的位元數已由激活值記憶體中原本的 7 位元（最低有效位元因期望值補償而固定為 1）擴增為多位元，因此必須將結果重新量化回 7 位元。考量已在期望值補償中將第 8 位元固定為 1，故在重新量化時可直接採用截斷方式，而無需再進行四捨五入操作。

3.4.9 張量處理器系統控制器 (TPU System Controller)

本研究通過圖二十二所示之有限狀態機 (Finite State Machine, FSM) 去產生對應的控制信號以控制整個張量處理器架構的流程。

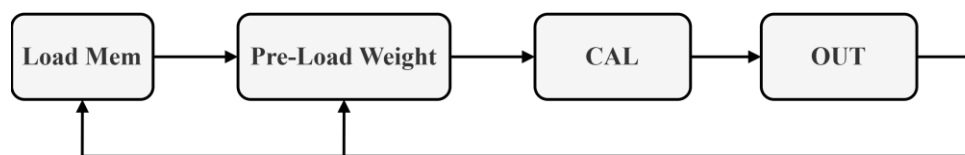
首先在張量處理器啟動時，有限狀態機會進入 Load Mem 的狀態，此階段負責等待權重和激活值資料完全寫入記憶體。寫入完成後，由於本研究採用權重固定資料流的方式，因此有限狀態機進入到 Pre-Load Weight 階段，會將權重預先載入到處理單元。若以 $n \times n$ 的脈動陣列為例，其將權重傳遞到最底端需要 n 個 c 時脈週期。

隨後，有限狀態機進入 CAL 階段，開始執行主要乘加運算，激活值會通過輸入緩衝器以 45 度角傳播到整個脈動陣列中，此階段的週期計算如下：

- (1) 需要 $n - 1$ 個時脈週期進行輸入緩衝
- (2) 需要 n 個時脈週期將所有激活值傳入脈動陣列中
- (3) 需要 $n - 1$ 個時脈週期將最後一個激活值傳播到脈動陣列最末端

因此，CAL 階段共需要 $3n - 2$ 個時脈週期，也就是說，完成一次矩陣運算需要 $4n - 2$ 個時脈週期。最後有限狀態機轉換到 OUT 階段。該階段將累加器的結果輸出，然後執行 ReLU 跟重新量化操作，最後存入統一緩衝器，共需要 n 個時脈週期。

因此，只要根據上述計算出來的時脈週期，我們就可以準確調整有限狀態機的狀態去控制整個張量處理器架構的流程



圖二十二：TPU System Controller 有限狀態機示意圖

3.5 電路模擬

在本章節中，將透過 Intel ModelSim 對 WPU、RPE、CPE 以及整體張量處理器架構進行電路模擬與驗證，以確保本研究所設計之架構在實際硬體實現中的行為正確無誤。所有相關的硬體描述語言 (Verilog HDL) 程式碼均已公開於 [GitHub 平台](#)，供後續研究與驗證參考，除此之外，本研究以 8 x 8 個 RPE 所組成的縮減精度脈動陣列與 8 x 3 個 CPE 組成的補償精度脈動陣列作為範例實現。

3.5.1 權重預處理單元電路模擬

圖二十三為權重預處理單元電路模擬範例，可以發現位址 15 的資料是 MSR-4 的權重，因此權重預處理單元將其高位元連續的四個 1 壓縮成一個，捨棄掉 LSB 並於 MSB 加上位移位元後寫入到權重記憶體中。除此之外，由於我們是實現 8 x 8 的脈動陣列，當位址指到 8 個倍數時補償記憶體就需要換行，因此可以看到 Change_col 被拉高，補償記憶體寫入位置調到 6。而位址 16 的資料是 Non-MSR-4 的權重，因此將低位元部分做截斷，MSB 加上位移位元後寫入到權重記憶體，而被截斷部分則是加上原資料的符號位元且捨棄 LSB 後寫入到權重記憶體中。

/tb_TPU/P0/Weight_Pre_Processing_Unit/clock	St0				
/tb_TPU/P0/Weight_Pre_Processing_Unit/rst	St0				
/tb_TPU/P0/Weight_Pre_Processing_Unit/Mem_Write	St1				
/tb_TPU/P0/Weight_Pre_Processing_Unit/Weight_Mem_Address_in	18	16	17	18	
/tb_TPU/P0/Weight_Pre_Processing_Unit/Weight	11110000	11110000	00110011	11110000	
/tb_TPU/P0/Weight_Pre_Processing_Unit/Reduced_Weight	01000	01000	10011	01000	
/tb_TPU/P0/Weight_Pre_Processing_Unit/Compensation_Weight	0001	0011	0001		
/tb_TPU/P0/Weight_Pre_Processing_Unit/Compensation_out_valid	0				
/tb_TPU/P0/Weight_Pre_Processing_Unit/Compensation_Row	0	1	0		
/tb_TPU/P0/Weight_Pre_Processing_Unit/Weight_Mem_Address_out	17	15	16	17	
...TPU/P0/Weight_Pre_Processing_Unit/Compensation_Mem_Wr_Addr	7	4	6	7	
/tb_TPU/P0/Weight_Pre_Processing_Unit/Non_MSR_4	St0				
/tb_TPU/P0/Weight_Pre_Processing_Unit/Boundary_limit	0	1	0		
/tb_TPU/P0/Weight_Pre_Processing_Unit/Judge	01	01	00	01	
/tb_TPU/P0/Weight_Pre_Processing_Unit/change_col	St0				

圖二十三：權重預處理單元電路模擬範例

3.5.2 RPE 電路模擬

圖二十四為 RPE 電路模擬的範例一，該權重是 MSR-4 權重，因此其值會做期望值補償是 $10001 = -15$ ，其乘加運算結果如下圖所示。



圖二十四：RPE 電路模擬範例一

圖二十五為 RPE 電路模擬的範例二，該權重是 Non-MSR-4 權重，其值為 0110，做完乘法後運算結果需要向左位移 4 位元，加上前一級的處理單元的部分和輸入，也就是圖 n 的乘加結果。



圖二十五：RPE 電路模擬範例二

3.5.3 CPE 電路模擬

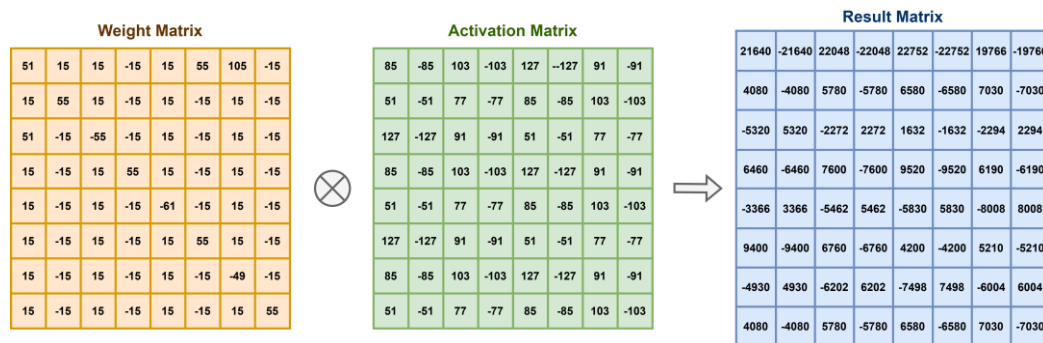
圖二十六為 CPE 的電路模擬範例，展示了權重和激活值做了 LSB 期望值補償後的乘加運算結果。



圖二十六：CPE 電路模擬

3.5.4 張量處理器電路模擬

圖二十七為張量處理器電路模擬的其中一個測試向量，展示了將權重矩陣與激活矩陣相乘後的結果矩陣。



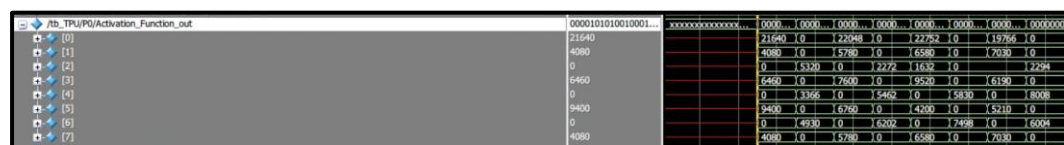
圖二十七：測試向量與矩陣運算結果

圖二十八為累加器記憶體的内部存儲的值，可以發現與圖二十八所計算出來的運算結果相同，顯示電路模擬正確。



圖二十八：累加器輸出

透過激活值函數後，矩陣中的負數元素將轉為零，如圖二十九所示。



圖二十九：激活值函數輸出

3.6 硬體開銷分析與超大型積體電路實現結果

本研究使用 Cell-Based Design 的 ASIC 設計流程實現所提出之張量處理器架構，使用製程為 TSMC Artisan 90 nm 1P9M 與標準元件資料庫 (Standard Cell Library) 並透過 Synopsys Design Compiler 完成邏輯合成、Synopsys IC Compiler II 完成自動佈局與繞線。

表五主要分析本研究提出的降低精度處理單元 (RPE) 與補償精度處理單元 (CPE) 和傳統處理單元 (PE) 的面積成本與功耗。通過計算可以發現在 256 x 256 的脈動陣列設計中，即使加入 256 x 3 的補償陣列，整體硬體開銷相較於傳統設計仍減少了 16.64%，代表我們利用補償陣列來換取模型準確度所帶來的硬體負擔很小，同時，由於計算量的下降，降低精度處理單元相比傳統處理單元的功耗下降了 0.1344mW。

表五：各處理單元的硬體成本、功耗比較

Type of PE	PE	RPE	CPE
Area	0%	-18.8%	-30.6%
Power (mW)	0.6743	0.5399	0.4327

根據表六可知，由於本研究新增了補償機制，因此輸入緩衝器會增加 1.25 倍的面積用於儲存列索引與輸出對應的激活值給補償陣列，但通過表七可以知道，其實輸入緩衝器與脈動陣列大小佔比相差甚遠，因此不會對整體的硬體開銷產生過多的負擔。

表六：原始與本研究所提出之輸入緩衝器硬體成本比較

Type of Input Buffer	Original	Proposed
Area	0%	+125%

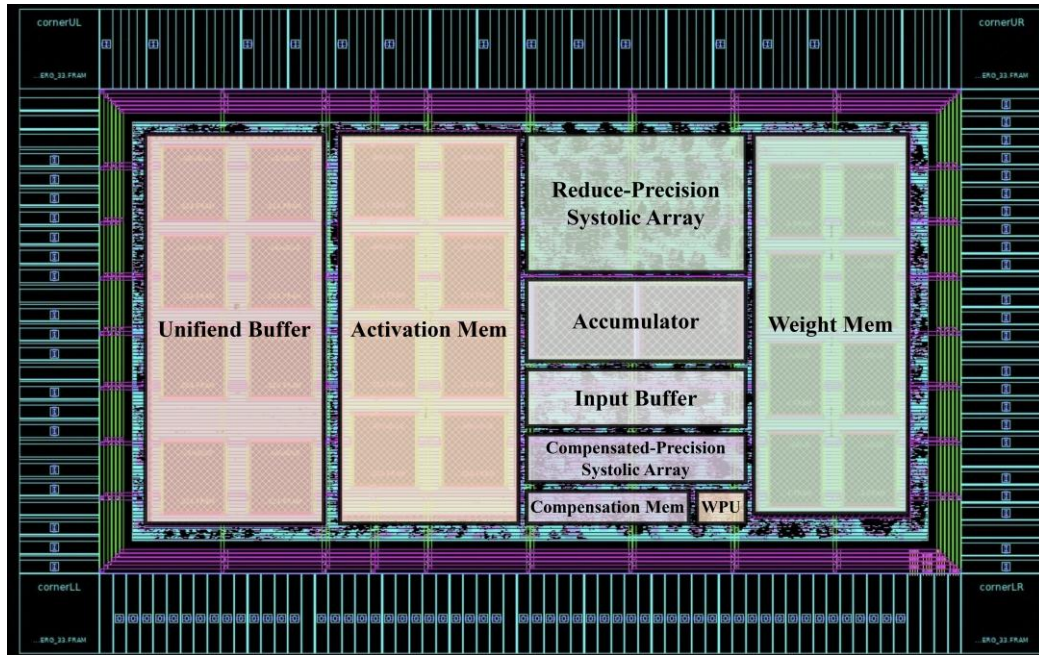
除了處理單元以外，由於對於權重採取預先壓縮的策略，搭配上期望值補償的概念，這使得權重記憶體每個資料所需的位元數變為 5 個位元，而激活值記憶體所需位元數變為 7 個位元，雖然增加了一個每個資料所需位元數為 4 位元的補償記憶體，但以 256 x 256 的脈動陣列為例，其晶片上記憶體 (On-chip Memory) 依然可以減少約 18.46%。

表七為本研究所提出之張量處理器的超大型積體電路實現結果，電路操作頻率可以達到 233 MHz，其中激活記憶體與權重記憶體分別為 56 KB 與 40 KB，加上一個 56 KB 的統一緩衝器，總共佔整體電路的 91.54 %，在邏輯電路的部分，張量處理器的縮減脈動陣列與累加器分別佔整體電路的 5.81 % 與 1.25 %，額外加入的補償精度脈動陣列佔整體電路的 0.93 %，而其餘的部分則是權重預處理單元與激活函數轉換元件等周邊電路。

表七：本研究所提出之張量處理器的超大型積體電路實現結果

DESIGN SPECIFICATIONS	ITEM
Process Technology	TSMC 90 nm 1P9M
Frequency	233 MHz
Weight Memory Size	40 KB
Activation Memory Size	56 KB
Compensation Memory Size	48 B
Unified Buffer Size	56 KB
Total Memory Gate Count	1188736 (91.53%)
Input Buffer Gate Count	7629 (0.39%)
TPU System Controller Gate Count	955 (0.05%)
WPU Gate Count	545 (0.03%)
Reduced-Precision SA Gate Count	114277 (5.81%)
Compensated-Precision SA Gate Count	18381 (0.93%)
Accumulator Gate Count	24633 (1.25%)
Activation Function Gate Count	313 (0.02%)
Core Area	3,000,220 μm^2
Chip Area	4,170,448 μm^2

圖三十為本研究所提出之張量處理器的佈局圖，供電電壓為 1.5 伏特，核心面積的大小為 $3,000,220 \text{ um}^2$ ，而晶片面積大小為 $4,170,448 \text{ um}^2$ ，圖中最左方是統一緩衝器，再來是激活值記憶體，最右方則是權重記憶體，中間的部分包含了 8×8 的縮減精度脈動陣列、累加器、輸入緩衝器、 8×3 的補償精度脈動陣列、補償記憶體、以及權重預處理單元與其他剩餘電路。



圖三十：本研究所提出之張量處理器的佈局圖

3.7 準確度分析

表八顯示了在四種深度神經網路模型下，採用不同量化策略所造成的推論準確度差異。可以觀察到，若使用 RPTPU [3] 所提出的 RPE Type 1 架構，在較為複雜的模型 (如 ResNet 與 AlexNet) 中，推論準確度明顯下降，甚至出現嚴重的效能劣化。相較之下，RPE Type 2 透過引入期望值補償策略，有效修正了權重量化所造成的誤差，使上述複雜模型的準確度得以恢復至接近原始 INT8 量化的水準。進一步比較本研究所提出的架構，可以發現在較輕量的模型 (如 MLP 與 LeNet) 中，推論準確度甚至超越了傳統 INT8 量化結果。這主要歸因於本研究在 MSR-4 權重處理中結合了 Non-MSR-4 與期望值，使模型在保持低位元精度的同時，仍能維持高準確度。

綜上所述，本研究的量化與補償策略不僅能在深層與淺層網路架構中皆維持穩定表現，亦顯示出在低位元推論中具備更佳的可擴展性與泛用性，特別適合應用於邊緣運算及高效能推論加速器之設計中。

表八：不同量化策略之推論準確度分析

Precision / Model	MLP	LeNet	ResNet	AlexNet
Float 32	98.08%	98.01%	99.61%	99.56%
INT8	97.28%	97.97%	99.09%	99.45%
MSR-4 [3]	92.71%	89.20%	11.36%	19.27%
MSR-4 + EV [3]	97.29%	97.44%	98.96%	99.40%
MSR-4 + EV + COMP	97.34%	98.00%	98.96%	99.40%

3.8 性能分析

本研究架構由於透過壓縮權重字長的方式縮減每個處理單元的計算量，並且補償架構是平行化處理，並不會額外花費時間計算，因此其每秒兆次運算 (Tera Operations Per Second ,TOPS)相較於傳統 INT8 量化的架構，快了 6.54，如表九所示。

表九：TPOS 比較

Type of TPU	Original TPU	Proposed TPU
TOPS (256 x 256)	22.02	28.56

四、建議與結論

本研究針對深度神經網路權重於訓練後所呈現的連續重要位元分布特性，提出一套兼具低硬體成本與高推論精度之張量處理器架構。透過對 MSR-4 權重之分布分析，證實多數權重在高位元端具備可壓縮性，進而能以固定長度的縮減字長進行有效表示；同時搭配少量補償陣列以修正 Non-MSR-4 權重截斷所造成的精度損失。此外，本研究也採用論文 [2] 所提出的期望值補償策略，以彌補訓練後量化為 INT8 所產生的誤差，有效抑制低位元截斷帶來的偏差性累積，使低精度運算仍能維持與高精度量化相近的推論結果。因此，本研究透過權重字長壓縮以降低硬體成本，並分別利用補償陣列與期望值補償修正兩類誤差來源，在低硬體資源條件下依然實現高推論準確度。

在推論效能方面，本研究架構於 MLP 與 LeNet 等模型上皆達到優於傳統 INT8 量化之準確度，而在 ResNet 與 AlexNet 等較大規模模型上亦保持高度一致的推論結果，顯示本架構在不同深度與複雜度的神經網路中均具穩定性與實用性。此結果說明，MSR-4 壓縮與期望值補償之整合量化方法，能有效支援深度模型於極低位元環境中的高可靠度推論。

在硬體實現方面，本研究以 TSMC 90 nm 製程進行 ASIC 合成，結果顯示本架構在加入補償陣列後，整體脈動陣列面積仍較傳統 INT8 設計減少 16.64%，晶片上記憶體需求亦降低 18.46%，並因乘加單元計算複雜度下降而使整體性能提升 6.54 TOPS。此外，補償陣列可與主體陣列共用累加器與記憶體頻寬，不需額外硬體資源，進一步達成整體架構之輕量化與高效能。

綜上所述，本研究驗證了以權重統計特性為基礎之資料壓縮與補償方法在硬體感知量化（hardware-aware quantization）中的可行性，並提出一套適用於邊緣運算場景的低精度張量處理器架構。此架構不僅有助於降低 AI 加速器之面積、功耗與記憶體帶寬需求，亦為未來多格式資料表示（如 MSFP、BFP）與混合精度推論技術之發展奠定基礎。期望本研究所建立之設計理念，能促進低精度深度學習硬體於更廣泛應用中落地，並強化 AI 加速器在未來邊緣與分散式推論環境中的整體效能與競爭力。

五、未來發展方向

本專題研究截至目前為止，提出並驗證了一種基於連續重要位元特性之低成本人工智慧加速器架構。透過一些補償機制，我們證實了在大幅度縮減權重位寬的同時，仍能維持優異的推論準確度。然而，面對人工智慧領域中參數量更大、動態範圍更廣的大型語言模型，現有的定點數量化仍有優化空間。為了使本架構能適應未來更高效能的邊緣運算場景，我們計畫將現有的統計壓縮技術，進一步演進為更具通用性與標準化的兩個方向：

5.1 Block Floating Point (BFP)

Block Floating Point 是針對運算資料流與處理單元內部的深度優化。BFP 結合了定點數運算的低硬體成本與浮點數運算的高動態範圍優勢，其特點在於資料區塊內的數值共用指數，而尾數 (Mantissa) 則保持為定點數運算，若尾數能觀察出 MSR 之特性，會與本研究的設計理念高度契合。

Block Floating Point 硬體開銷分析：

現在有以下四組資料 x_1 、 x_2 、 x_3 、 x_4

$$x_1 = 1.575 * 2^5 / x_2 = 1.525 * 2^5 / x_3 = 0.725 * 2^5 / x_4 = 0.175 * 2^5$$

這四個數值的共享指數為 5，只有尾數有不同，如果我們以傳統的 Floating Point 32 格式去儲存這四筆資料，每筆資料會要 32 bits 的儲存空間。

➔ *Total Bits for FP32* = $4 * 32 = 128 \text{ bits}$

但在這裡如果使用 Block Floating Point 的格式表示，我們可以只存儲一次共享指數，然後再分別儲存每個數據的尾數。假設指數以 8 bits 來表示，尾數以 23 bits 表示並在每個尾數前放入 1 bit 的符號為元 (共 24 bits)。

➔ *Total Bits for BFP* = $8 + 4 * 24 = 104 \text{ bits}$

根據初步運算可以看到 Block Floating Point 格式相較於 Floating Point 格式少了約 **18.75%** 的硬體開銷，展現了其在節省儲存空間上的優勢。這種數據儲存的方法除了能在硬體成本上面有不錯的表現外，最大的誘因在於其不需要複雜的浮點數運算單元，用簡單的定點數運算移位即可完成，使整體功耗及運算時間減少。

而在使用 BFP 時須注意小數值被淹沒，這是 BFP 最致命的缺點。如果一個區塊內同時存在一個大數和一個小數，為了對齊大數的指數，小數必

須向右移位很多次。在有限的尾數位寬下，小數的所有有效位元可能會被移出邊界而變成 0，這種現象可能會影響模型收斂或推論精確度，也是一項重大的挑戰。

5.2 Microsoft Floating Point (MSFP) [11]

Microsoft Floating Point 與 Block Floating Point 在核心原理上具備極高相似度，本質上是使用共用指數的概念，將複雜的浮點點積運算轉化為低成本的定點數乘加運算，從而讓硬體能以接近定點數的低面積與低功耗，實現浮點數的高動態範圍，期望解決邊緣人工智慧加速器面臨的記憶體頻寬瓶頸與計算效率挑戰，若我們能夠將現有壓縮概念納入 MSFP 的系統中，不僅可以降低更多的硬體成本，其浮點數的運算範圍也會使得推論準確度更加提高。

六、時間進度表

月 份 (月) 項目內容	三	四	五	六	七	八	九	十	十一
論文研讀	3/1	4/21						10/6	11/2
架構設計		4/15		6/2					
DNN 模擬			4/20		6/15				
RTL 程式撰寫					6/12			9/16	
合成&分析							9/10		11/9
期初報告製作		4/15		5/25					
期末報告製作								10/1	
進 度 百 分 比 (%)	10	15	25	40	50	75	85	95	100

七、參考資料

- [1] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, N. Boden, A. Borchers, and R. Boyle, “In-Datacenter Performance Analysis of a Tensor Processing Unit,” in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, pp. 3–4, June 2017.
- [2] S. Hashemi, R. I. Bahar, and S. Reda, “DRUM: A Dynamic Range Unbiased Multiplier for Approximate Applications,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design*, pp. 419–420, Nov. 2015.
- [3] Mohammed E. Elbity, Peyton S. Chandarana, Brendan Reidy, Jason K. Eshraghian, and Ramtin Zand, “APTPU: Approximate Computing Based Tensor Processing Unit,” *IEEE Trans. Circuits Syst. I*, vol. 69, no. 12, pp. 5135–5144, Dec. 2022.
- [4] Shyue-Kung Lu and Yu-Xian Huang, “Weight-Aware and Reduced-Precision Architecture Designs for Low-Cost AI Accelerators,” pp. 42-45,
- [5] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "SCALE-Sim: Systolic CNN Accelerator Simulator," arXiv preprint arXiv:1811.02883, 2018.
- [6] T. F. Hsieh, J. F. Li, J. S. Lai, C. Y. Lo, D. M. Kwai, and Y. F. Chou, “Refresh Power Reduction of DRAMs in DNN Systems Using Hybrid Voting and ECC Method,” in *Proc. IEEE International Test Conference in Asia*, pp. 41–46, Sep. 2020.
- [7] H. Ramchoun, M. A. Idrissi, Y. Ghanou, and M. Ettaouil, “Multilayer Perceptron: Architecture Optimization and Training with Mixed Activation Functions,” in *Proc. of the 2nd Int. Conf. on Big Data, Cloud and Applications*, pp. 1–6, Mar.2017.

- [8] M. Kayed, A. Anter, and H. Mohamed, “Classification of Garments from Fashion MNIST Dataset Using CNN LeNet-5 Architecture,” International Conference on Innovative Trends in Communication and Computer Engineering, pp. 238–243, Feb. 2020.
- [9] M. F. Haque, H. Y. Lim, and D. S. Kang, “Object Detection Based on VGG with ResNet Network,” in Proc. International Conference on Electronics, Information, and Communication, pp. 1–3, Jan 2019.
- [10] T. Shanthi and R. S. Sabeenian, “Modified Alexnet Architecture for Classification of Diabetic Retinopathy Images,” Computers & Electrical Engineering, pp. 5664, Jun. 2019.
- [11] S. Chatterjee et al., “Pushing the Limits of Narrow Precision Inferencing at Cloud Scale with Microsoft Floating Point,” Microsoft Research, Tech. Rep., 2023.
[\[Online\]](#)

八、工作分配

項目	林明宏	蕭家宏
論文研讀	✓	✓
DNN 模擬	✓	
架構設計	✓	
RTL 撰寫與電路模擬	✓	
邏輯合成	✓	✓
實驗結果分析	✓	✓
自動佈局與繞線 (APR)	✓	
專題計劃書製作	✓	✓
專題成果報告書製作	✓	✓
海報製作	✓	✓
成果展示簡報製作	✓	✓
工作佔比	70 %	30 %

九、經費表

項目種類	數量(個)	單價(元)
電腦	2	0
Intel Modelsim	2	0
AMD Vivado 2025.1	2	0
Synopsys Design Compiler	1	0
Synopsys Memory Compiler	1	0
Synopsys IC Compiler II	1	0
總價	0	

本專題皆使用實驗室現有工具及免費軟體進行研究，故此次研究並無任何花費。
(注：AMD Vivado 2025.1 用於 ASIC 合成與佈局前先進行 FPGA 驗證時使用)